Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Компьютерного проектирования
Кафедра Инженерной психологии и эргономики
Дисциплина Интерфейсы информационных систем

Лабораторная работа №25
«Перенос веб-приложения на Nuxt.js и гибридный рендеринг»

Студент группы 310102 _____ Кадол А.П.
(подпись)

Руководитель _____ Прудник А.М.
(подпись)

Минск 2026

Цель работы: перенести блог Folio на Nuxt.js и показать гибридный рендеринг: серверную сборку главной страницы и статическую генерацию страниц статей.

Исходные данные. Вариант. Блоговая платформа: главная страница со списком статей должна собираться на сервере при каждом запросе, страница отдельной статьи — статически для всех существующих материалов.

Исходное приложение — блог Folio на Vue. В каталог `nuxt-app` перенесены карточка статьи, список и блок комментариев. Данные трёх статей лежат в `utils/articles.js`, обложки и значки — в `public`. Маршруты создаёт файловый роутинг: `pages/index.vue` и `pages/posts/[slug].vue`. Переходы между страницами выполняются компонентом `NuxtLink`.

Теоретические сведения. SSR отдаёт готовый HTML при каждом запросе, поэтому на главной видны свежие публикации и время генерации. SSG собирает HTML заранее, во время `npm run build`, и сервер отдаёт уже готовый файл. В Nuxt 3 данные для обоих режимов получает `useAsyncData`: это аналог `asyncData` и функций `getServerSideProps` / `getStaticProps`. Список адресов, которые нужно сгенерировать, задаётся в `nitro.prerender.routes` — аналог `getStaticPaths`.

Ход выполнения работы. Стратегии закреплены в конфигурации. Корень не пререндерится, маршруты статей пререндерятся явно, обход ссылок отключён, чтобы главная не попала в статику.

```
routeRules: {
  '/': { ssr: true, prerender: false },
  '/posts/**': { prerender: true },
},
nitro: {
  prerender: {
    crawlLinks: false,
    failOnError: true,
    routes: [
      '/posts/morning-pages',
      '/posts/when-you-dont-want-to-write',
      '/posts/editing-aloud',
    ],
  },
},
```

На главной `useAsyncData` вызывается при запросе. В ответ кладётся текущее время и список статей. Метка на странице показывает, что HTML собран заново.

```
const { data } = await useAsyncData('folio-home', () => {
  return {
    generatedAt: new Date().toISOString(),
    articles: getArticles(),
  }
})
```

Страница статьи читает `slug` из маршрута. Если материала нет, возвращается ошибка 404. Время сборки попадает в HTML один раз и дальше не меняется.

```
const route = useRoute()
const { data } = await useAsyncData(`post-${route.params.slug}`, () => {
  const article = getArticle(String(route.params.slug))
  if (!article) {
    throw createError({ statusCode: 404, statusMessage: 'Статья не найдена' })
  }
  return { article, builtAt: new Date().toISOString() }
})
```

Сборка `npm run build` завершилась успешно: пререндерены три HTML-страницы и полезные нагрузки к ним. Запуск — `npm run start`, сервер слушает порт 3000.

Проверка по исходному HTML. Два подряд запроса главной вернули разные метки: `2026-10-21T05:29:04.694Z` и `2026-10-21T05:29:04.707Z`. Страница `/posts/morning-pages` и файл `.output/public/posts/morning-pages/index.html` содержат одну и ту же метку `2026-10-21T05:28:27.173Z`. В теле ответа уже есть заголовок статьи и абзацы, атрибут `data-ssr` равен `true`. Значит, контент пришёл с сервера, а не был дорисован пустым клиентским каркасом.

Таблица 1 – Страницы и стратегии рендеринга

Маршрут	Режим	Зачем
`/`	SSR	Свежая лента при каждом запросе
`/posts/[slug]`	SSG	Готовый HTML трёх статей

Главная страница после production-запуска показана на рисунке 1. В исходном HTML ответа сервера видны заголовок Folio, время генерации и карточка статьи (рисунок 2).

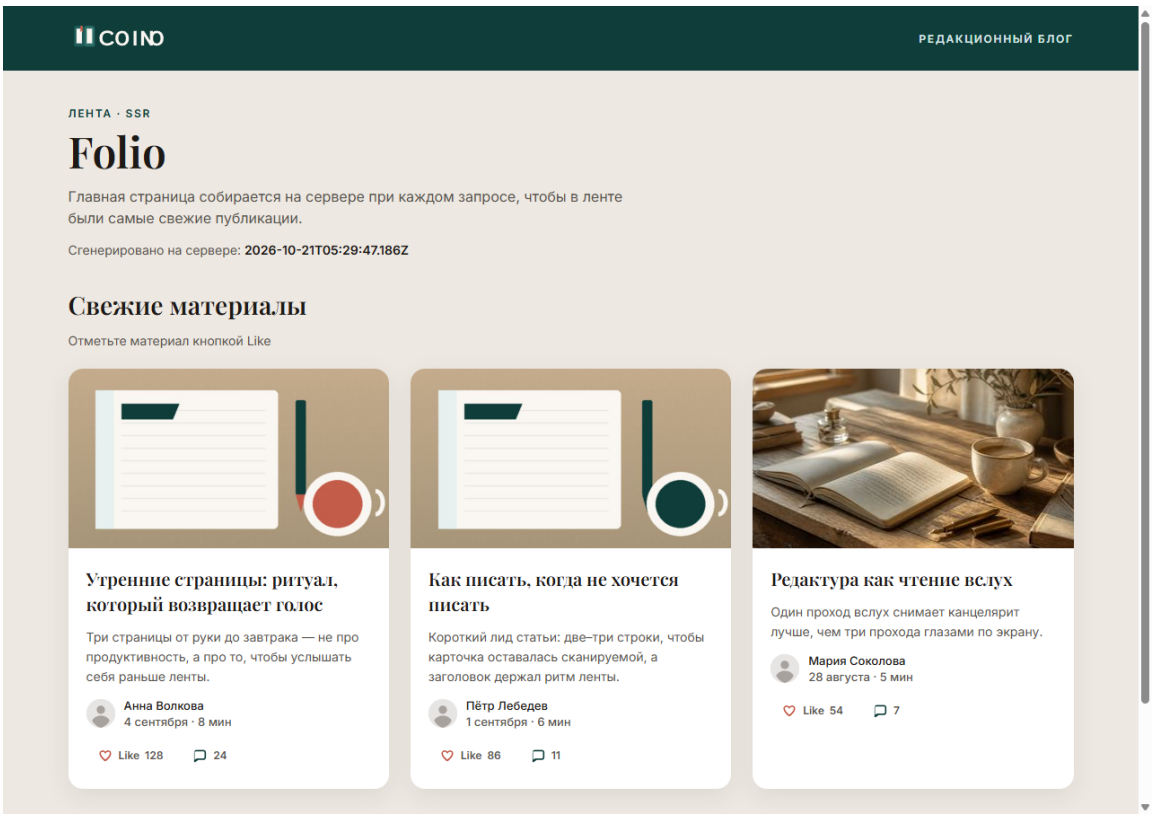


Рисунок 1 – Главная страница, собранная на сервере

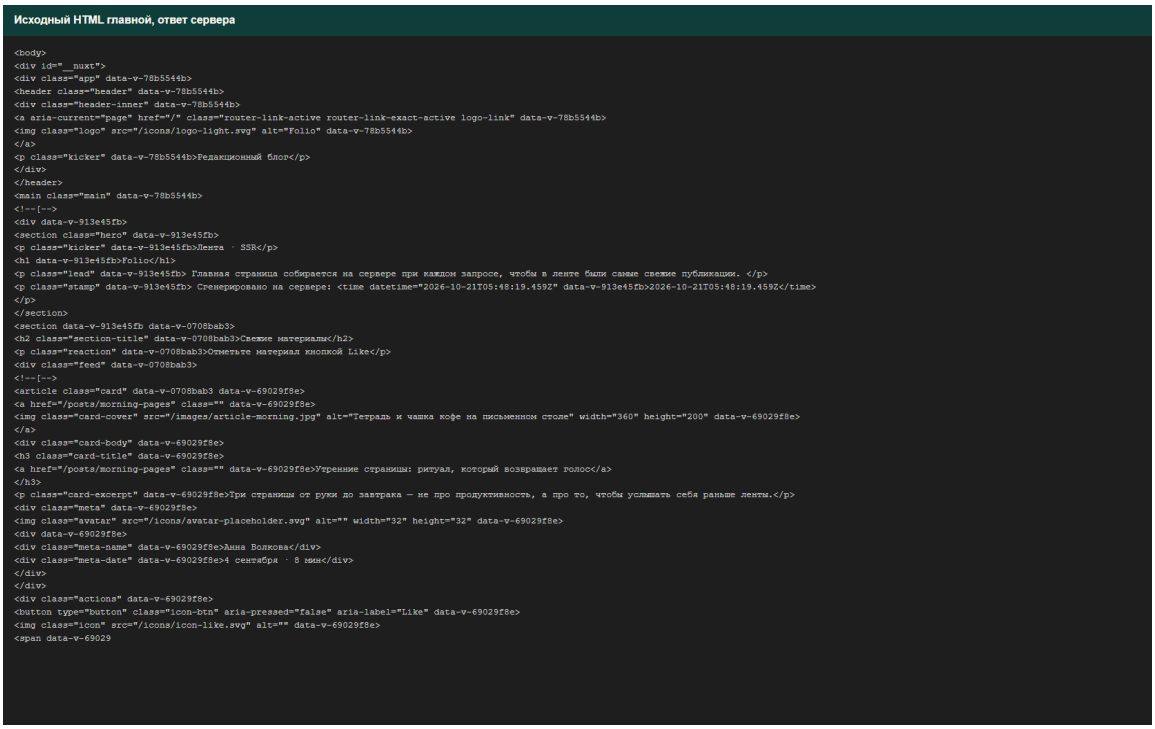


Рисунок 2 – Фрагмент HTML главной страницы из ответа сервера

Страница статьи `/posts/morning-pages` показана на рисунке 3. Метка сборки совпадает с файлом, полученным при `npm run build`. Тот же текст есть в исходном HTML (рисунок 4).

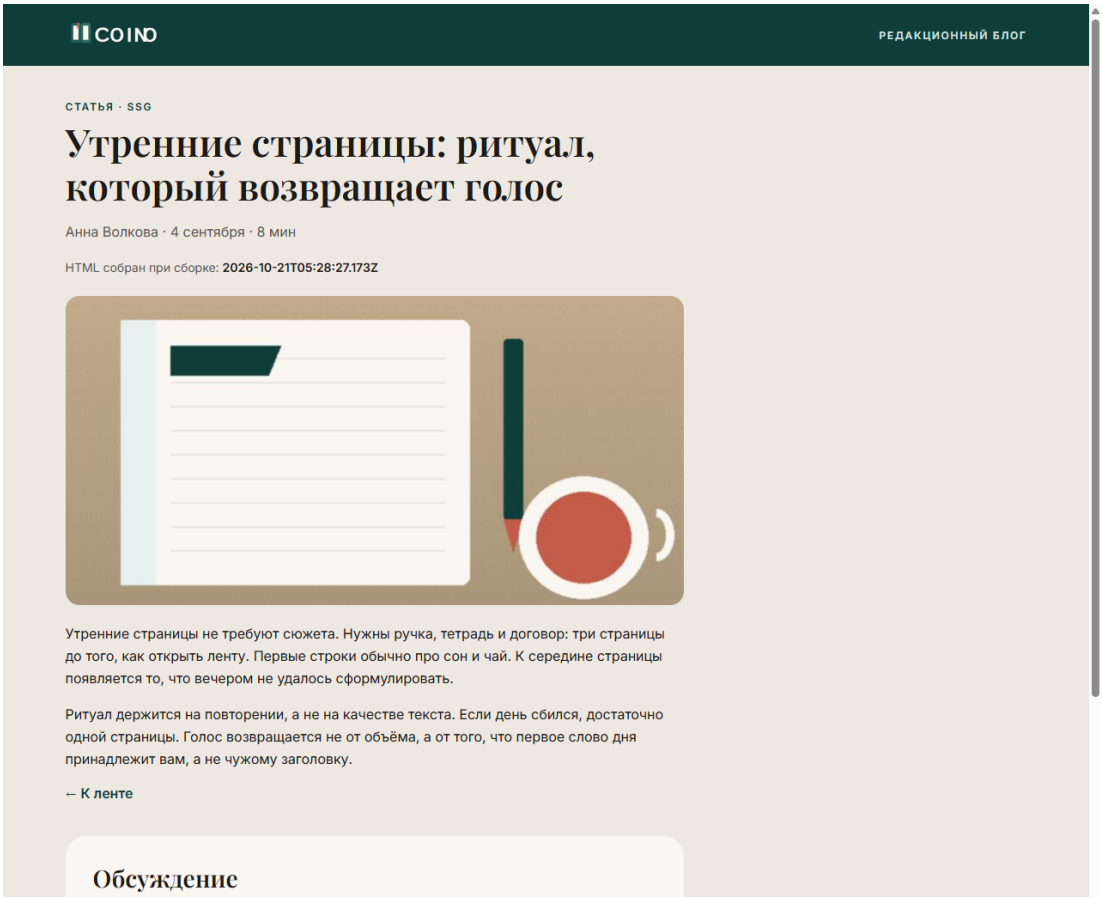


Рисунок 3 – Статическая страница статьи

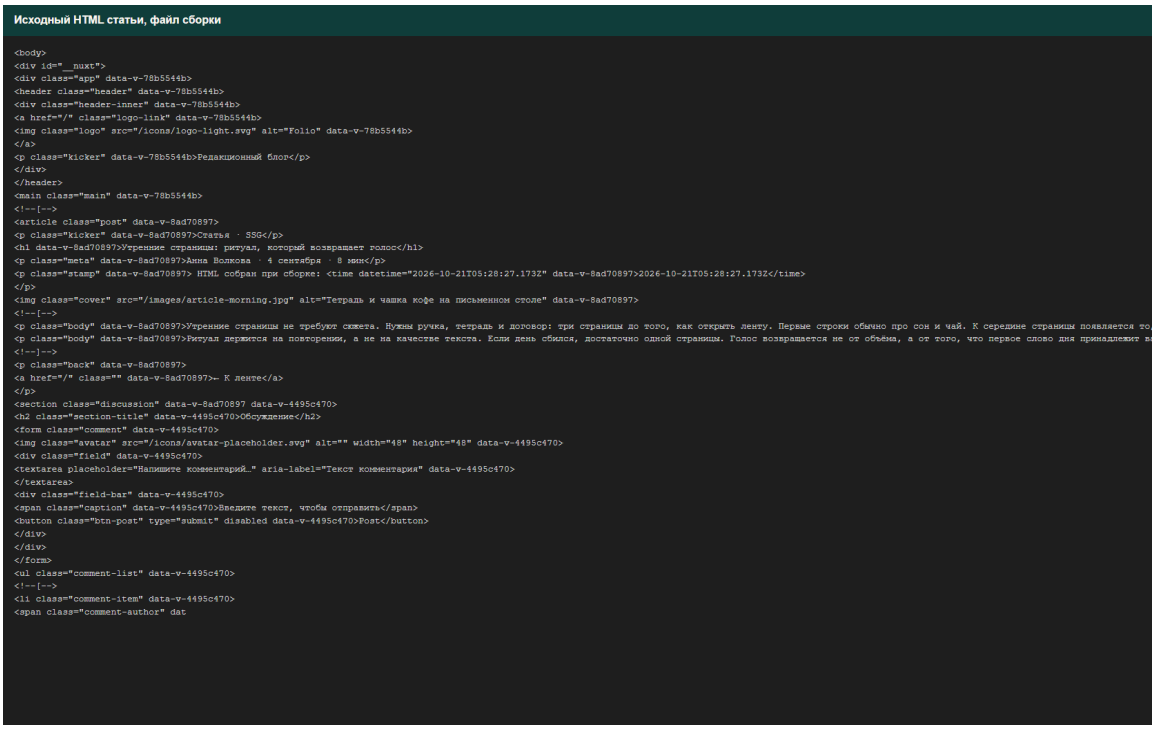


Рисунок 4 – Фрагмент HTML статьи из файла сборки

Вывод: блог Folio перенесён на Nuxt.js. Главная страница рендерит-

ся на сервере при каждом запросе, страницы трёх статей генерируются при сборке. Исходный HTML содержит тексты материалов, production-сборка и запуск проходят командами `npm run build` и `npm run start`.